

LM Pizzeria: Phase 1 Project Report

React Frontend Web Application

Course: CSCI 426 - WEH Advanced

Instructor: Prof. Fouad Najem

Semester: Spring 2026

Student Name: Lamita Masry

Date: January 2026

Live Demo: <https://lmpizzeria.netlify.app/>

GitHub Repository: <https://github.com/Lamitamassry/lm-pizzeria>

Abstract

This report documents the development of LM Pizzeria, a modern restaurant ordering web application built with React. The project implements a complete user interface for browsing menu items, managing a shopping cart, and placing orders. The application features 10 distinct pages including a dynamic routing system, responsive design using Tailwind CSS, and state management through React Context API. The frontend is deployed on Netlify and demonstrates modern web development practices including component-based architecture, client-side routing with React Router, and mobile-first responsive design principles.

1. Introduction

1.1 Project Overview

LM Pizzeria is a web-based restaurant ordering system that allows customers to browse a menu, add items to their cart, and complete orders online. The application focuses on providing an intuitive user experience with a clean, responsive interface that works seamlessly across all device sizes.

1.2 Project Objectives

The project was designed with the following key objectives:

- Build a modern, interactive frontend using React framework
- Implement client-side routing for multiple pages and views
- Create a responsive design that works on mobile, tablet, and desktop devices
- Develop reusable components following React best practices
- Manage application state effectively using Context API
- Deploy the application to a live production environment

1.3 Technology Choices

The project uses React 18.3.1 as the core framework due to its component-based architecture, efficient rendering with virtual DOM, and extensive ecosystem. Vite was selected as the build tool for its fast development server and optimized production builds. Tailwind CSS provides utility-first styling that enables rapid UI development while maintaining consistent design. React Router DOM handles client-side navigation without page reloads, creating a smooth single-page application experience.

2. System Design

2.1 Application Architecture

The application follows a standard React single-page application (SPA) architecture:

Presentation Layer:

- React components for UI rendering
- Tailwind CSS for styling
- React Router for navigation

State Management:

- React Context API for global cart state
- Component-level state with useState hook
- React Router for URL state

Data Layer:

- Static menu data in JavaScript module
- localStorage for cart persistence (future enhancement)

2.2 Component Structure

The application is organized into a hierarchical component structure:

Layouts:

- MainLayout: Wraps all pages with common Navbar and Footer

Pages (10 total):

- Home: Landing page with hero section and featured items
- About: Restaurant information and story
- Menu: Complete menu with category filtering
- MenuItemDetail: Dynamic page for individual menu items
- Contact: Contact form and location details
- Cart: Shopping cart and checkout interface
- OrderConfirmation: Order success page
- Login: User authentication (Phase 2)
- Signup: User registration (Phase 2)

- Orders: Order history (Phase 2)

Reusable Components:

- Navbar: Responsive navigation header
- Footer: Site footer with links
- MenuItemCard: Reusable card for displaying menu items

Context:

- CartContext: Global state management for shopping cart
-

3. Page-by-Page Description

3.1 Home Page

The landing page features a hero section with a call-to-action button, followed by a showcase of featured menu items. It uses a responsive grid layout that adapts from single column on mobile to multi-column on larger screens.

Key Features:

- Hero banner with background image
- View Menu call-to-action button
- Featured items display
- Responsive grid layout

3.2 About Page

Presents the restaurant's story, mission, and team information. The page uses a two-column layout on desktop that stacks on mobile devices.

Key Features:

- Restaurant history and values
- Team member profiles
- Responsive text and image layout

3.3 Menu Page

Displays the complete menu organized by categories (Pizzas, Appetizers, Salads, Desserts, Beverages). Users can filter items by category using navigation tabs.

Key Features:

- Category filtering
- Grid display of menu items
- MenuItemCard component reuse
- Add to cart functionality

3.4 Menu Item Detail Page

A dynamic route that displays detailed information for a specific menu item based on the URL parameter. This page demonstrates React Router's dynamic routing capabilities.

Key Features:

- Dynamic URL parameter handling
- Detailed item information
- Larger image display
- Quantity selector
- Add to cart with custom quantity

3.5 Contact Page

Provides a contact form for customer inquiries and displays the restaurant's location information.

Key Features:

- Contact form with validation
- Restaurant address and hours
- Responsive form layout

3.6 Cart Page

Displays items in the shopping cart with options to update quantities or remove items. Includes a checkout form for order placement.

Key Features:

- Cart items display
- Quantity adjustment
- Remove item functionality
- Order type selection (Dine-in, Takeaway, Delivery)
- Checkout form with conditional address field
- Order total calculation

4. Key Technologies

4.1 React 18.3.1

React provides the foundation for building the user interface through reusable components. The project uses modern React features including:

- Functional Components: All components are written as functions
- Hooks: `useState`, `useContext`, `useNavigate`, `useParams`
- JSX: JavaScript XML syntax for component templates

4.2 React Router DOM 6.26.0

React Router enables client-side routing, allowing navigation between pages without full page reloads. This creates a smooth, app-like experience.

Key Features Used:

- Nested routes for layout wrapping
- Dynamic route parameters
- Programmatic navigation with `useNavigate`
- Link components for navigation

4.3 Tailwind CSS 3.4.11

Tailwind CSS is a utility-first CSS framework that enables rapid UI development through composable utility classes. This approach reduces the need for custom CSS and promotes consistency.

Benefits:

- Mobile-first responsive design
- Consistent spacing and sizing
- No naming conflicts
- Smaller CSS bundle (purges unused styles)

4.4 Vite 5.4.2

Vite is a modern build tool that provides fast development experience with hot module replacement and optimized production builds.

Development Features:

- Instant server start
- Fast hot module replacement
- Optimized dependency pre-bundling

Production Features:

- Code splitting
- Asset optimization
- Tree shaking for smaller bundles

5. State Management

5.1 Cart Context

The shopping cart state is managed globally using React Context API, allowing any component to access and modify the cart without prop drilling. The implementation includes methods for adding items, removing items, updating quantities, and calculating totals.

Core Functionality:

- addToCart: Adds or increments items in the cart
 - removeFromCart: Removes items from the cart
 - updateQuantity: Modifies item quantities
 - clearCart: Empties the entire cart
 - cartTotal: Calculates total price
-

6. Responsive Design Implementation

6.1 Mobile-First Approach

The application uses a mobile-first design strategy, where styles are initially defined for mobile devices and enhanced for larger screens using Tailwind's responsive breakpoints.

Breakpoints:

- sm: Small devices (640px and up)
- md: Medium devices (768px and up)
- lg: Large devices (1024px and up)
- xl: Extra large devices (1280px and up)

6.2 Responsive Navigation

The Navbar component adapts its layout based on screen size:

- Mobile: Hamburger menu icon that toggles a vertical menu
- Desktop: Horizontal navigation links

6.3 Responsive Grid Layouts

Menu and content grids adapt from single column on mobile to multi-column on larger screens, ensuring optimal viewing experience across all devices.

7. Version Control and Deployment

7.1 Git Version Control

The project uses Git for version control with a repository hosted on GitHub. The commit history documents the development process from initial setup through feature implementation.

Repository: <https://github.com/Lamitamassry/lm-pizzeria>

Branch Strategy:

- main: Production-ready code
- Feature branches for major changes (if applicable)

7.2 Netlify Deployment

The frontend is deployed on Netlify, providing a live, publicly accessible URL.

Deployment Configuration:

- Build Command: `npm run build`
- Publish Directory: `dist/`
- Live URL: <https://impizzeria.netlify.app/>

Deployment Process:

1. Code pushed to GitHub repository
 2. Netlify detects changes (if auto-deploy enabled)
 3. Runs build command (`vite build`)
 4. Deploys static files from `dist/` folder
 5. Application accessible at live URL
-

8. Technical Implementation Details

8.1 Dynamic Routing

The `MenuItemDetail` component demonstrates React Router's dynamic routing capabilities by using the `useParams` hook to access URL parameters. The component includes error handling for non-existent items, displaying a "not found" message with navigation options.

8.2 State Management Implementation

The `CartContext` provider is implemented at the application root level, allowing all components to access cart functionality through the `useCart` hook. This eliminates prop drilling and provides a clean, maintainable architecture.

8.3 Component Reusability

The `MenuItemCard` component is reused throughout the application wherever menu items need to be displayed, reducing code duplication and ensuring consistent styling and behavior across pages.

9. Challenges and Solutions

9.1 State Management Across Components

Challenge: Multiple components needed access to cart data without excessive prop drilling.

Solution: Implemented React Context API to provide global cart state accessible from any component using the `useCart` hook.

9.2 Dynamic Routing with Parameter Validation

Challenge: Handle cases where users navigate to non-existent menu item IDs.

Solution: Added validation in MenuItemDetail component to check if the item exists, displaying a "not found" message with navigation back to menu if the item doesn't exist.

9.3 Responsive Design Consistency

Challenge: Ensuring consistent layout across various screen sizes.

Solution: Used Tailwind CSS's mobile-first approach with systematic breakpoints, testing on multiple device sizes during development.

10. Future Enhancements

While the current implementation satisfies all Phase 1 requirements, potential future improvements include:

- Cart Persistence: Save cart data to localStorage so items persist across browser sessions
 - Search Functionality: Add search bar to filter menu items by name or ingredients
 - User Reviews: Allow customers to rate and review menu items
 - Dietary Filters: Add tags for vegetarian, vegan, gluten-free options
 - Image Optimization: Implement lazy loading and responsive images for better performance
 - Accessibility: Enhance keyboard navigation and screen reader support
-

11. Conclusion

The LM Pizzeria frontend application successfully demonstrates modern React development practices through a complete, production-ready web application. The project exceeds Phase 1 requirements with 10 pages (versus the 5 minimum), implements dynamic routing effectively, and provides a fully responsive user experience across all device sizes.

The application architecture follows React best practices with reusable components, efficient state management through Context API, and clean separation of concerns. The use of Tailwind CSS enables rapid UI development while maintaining design consistency, and Vite provides an excellent development experience with fast builds and hot module replacement.

The deployment to Netlify makes the application publicly accessible, demonstrating the complete development lifecycle from local development to production deployment. The codebase is well-organized, maintainable, and positioned for future enhancements including the Phase 2 backend integration.

12. Development Process and Learning Resources

Resources and Tools

This project was developed using official documentation, best practices guides, and modern development tools.

Development Tools: Visual Studio Code, Chrome DevTools, Git

Learning Resources:

1. Component Architecture: React official documentation for component structure, hooks usage, and state management patterns
2. Debugging Process: Chrome DevTools for debugging React Router configuration and identifying state management issues
3. Code Quality: JavaScript and React style guides for component organization and code readability improvements
4. Documentation: React documentation and academic writing resources for technical report structure and code comments
5. Best Practices: Official React documentation for patterns, Web Content Accessibility Guidelines (WCAG), and performance optimization techniques

Student Implementation:

- All business logic and feature requirements were defined by the student
- Code was developed iteratively using official documentation as reference
- UI/UX design decisions were made by the student based on modern design principles
- Project architecture and technology choices were student-driven based on course requirements
- All code was written with thorough understanding of its purpose and function
- Testing and deployment were performed by the student

13. References

- React Documentation: <https://react.dev/>
 - React Router Documentation: <https://reactrouter.com/>
 - Tailwind CSS Documentation: <https://tailwindcss.com/docs>
 - Vite Documentation: <https://vitejs.dev/>
 - MDN Web Docs: <https://developer.mozilla.org/>
 - GitHub Repository: <https://github.com/Lamitamassry/lm-pizzeria>
-

Appendix A: Project Structure

Project directory structure:

```
lami/
├── index.html
├── package.json
├── vite.config.js
├── tailwind.config.js
├── postcss.config.js
├── README.md
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   ├── index.css
│   ├── components/
│   │   ├── Navbar.jsx
│   │   ├── Footer.jsx
│   │   └── MenuItemCard.jsx
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── About.jsx
│   │   ├── Menu.jsx
│   │   ├── MenuItemDetail.jsx
│   │   ├── Contact.jsx
│   │   ├── Cart.jsx
│   │   ├── OrderConfirmation.jsx
│   │   ├── Login.jsx
│   │   ├── Signup.jsx
│   │   └── Orders.jsx
│   ├── context/
│   │   └── CartContext.jsx
│   ├── layouts/
│   │   └── MainLayout.jsx
│   ├── data/
│   │   └── menuData.js
│   ├── docs/
│   └── screenshots/
```

Appendix B: Package Dependencies

Core Dependencies:

- react: 18.3.1
- react-dom: 18.3.1
- react-router-dom: 6.26.0

Development Dependencies:

- @vitejs/plugin-react: 4.3.1
- autoprefixer: 10.4.20
- postcss: 8.4.47
- tailwindcss: 3.4.11
- vite: 5.4.2

Submitted by: Lamita Masry

Date: January 2026